

Lección 5: Code Smells

Detección y Análisis de Olores de Código

Agenda

- ¿Qué son los Code Smells?
 - Taxonomía de Code Smells
 - Code Smells más comunes en UI
 - Herramientas de Detección
-

¿Qué son los Code Smells?

“Un code smell es una señal superficial que habitualmente corresponde a un problema más profundo en el sistema” - Martin Fowler

Características:

-  Código funciona (no son bugs)
 -  Señal de alarma (problemas de diseño)
 -  Refactorizable
 -  Impacto futuro en mantenibilidad
-

Taxonomía de Code Smells

-  STRUCTURAL (Estructurales)
 - └ Long Method (método largo)
 - └ Large Class (clase grande)
 - └ Long Parameter List (lista larga de parámetros)

└─ Data Clumps (agrupaciones de datos)

BEHAVIORAL (Comportamiento)

- └─ Duplicate Code (código duplicado)
- └─ Switch Statements (declaraciones switch)
- └─ Lazy Class (clase perezosa)
- └─ Dead Code (código muerto)

OBJECT-ORIENTED (Orientados a Objetos)

- └─ Feature Envy (envidia de funcionalidad)
- └─ Inappropriate Intimacy (intimidad inapropiada)
- └─ Refused Bequest (herencia rechazada)
- └─ Middle Man (intermediario)

DATA (Datos)

- └─ Primitive Obsession (obsesión por primitivos)
- └─ Data Class (clase de datos)
- └─ Temporary Field (campo temporal)
- └─ Magic Numbers (números mágicos)

1. Magic Numbers

Problema:

```
//  ¿Qué significan estos números?  
if (quantity >= 5) {  
    return item.price * quantity * 0.1;  
}  
if (subtotal >= 100.0) {  
    return subtotal * 0.15;  
}
```

Solución:

```
//  Named Constants  
export const businessRules = {  
    bulkDiscount: { threshold: 5, percentage: 0.1 },  
    orderDiscount: { threshold: 100.0, percentage: 0.15 },  
} as const;
```

2. Duplicate Code

Problema:

```
// ❌ Validación repetida en 3 componentes
const handleQuantityChange = (value: string) => {
  const qty = parseInt(value);
  if (qty < 1) setError("Quantity must be at least 1");
  if (qty > 99) setError("Quantity cannot exceed 99");
  setQuantity(qty);
};
```

Solución:

```
// ✅ Custom Hook
export function useQuantityValidation(initial = 1) {
  const [quantity, setQuantity] = useState(initial);
  const [error, setError] = useState("");

  const updateQuantity = (newQty: number) => {
    // Validación centralizada
  };

  return { quantity, error, updateQuantity };
}
```

3. Long Parameter List (1/2)

Problema:

```
// ❌ Difícil de usar y recordar
function addItemToCart(
  items: CartItem[],
  productId: string,
  productName: string,
  productPrice: number,
  quantity: number,
  userId: string,
  sessionId: string,
  discount?: number,
```

```
): CartItem[];
```

3. Long Parameter List (2/2)

Solución:

```
//  Parameter Object  
interface AddToCartParams {  
  cart: CartItem[];  
  product: ProductItem;  
  quantity: number;  
  user: UserContext;  
}  
  
function addItemToCart(params: AddToCartParams): CartItem[];
```

4. Primitive Obsession

Problema:

```
//  Formateo de precio repetido en 8 lugares  
const formattedPrice = `>${product.price.toFixed(2)}$`;   
const total = `>${(price * quantity).toFixed(2)}$`;   
const discount = `>${discountAmount.toFixed(2)}$`;
```

Solución:

```
//  Helper function
export function formatPrice(amount: number): string {
  return amount.toFixed(2);
}

export function formatCurrency(amount: number): string {
  return `>${formatPrice(amount)}`;
}

// Uso
const formattedPrice = formatCurrency(product.price);
```

5. Dead Code

Problema:

- Funciones nunca llamadas
- Código comentado
- Imports no usados
- Código inalcanzable

Impacto:

- Aumenta tamaño del bundle
- Confunde a developers
- Debe mantenerse sin razón

Solución: ¡Eliminar! Git preserva la historia

6. Switch Statements

Problema:

```
//  Switch repetido en múltiples lugares
function calculateDiscount(discountType: string) {
  switch (discountType) {
    case "bulk": return /* ... */
    case "seasonal": return /* ... */
  }
}
```

```
        case "member": return /* ... */  
    }  
}
```

Solución: Strategy Pattern (siguiente lección)

7. Large Class/Component

Problema:

- Un componente hace demasiado
- 500+ líneas de código
- Múltiples responsabilidades
- Difícil de testear y reutilizar

Solución:

- Custom hooks para diferentes concerns
 - Componentes más pequeños y focalizados
 - Composition sobre monolitos
-

8. Feature Envy (1/2)

Problema:

```
// ✗ Usa más de otra clase que de sí misma  
class CartCalculator {  
    calculateItemTotal(item: CartItem): number {  
        let total = item.product.price * item.quantity;  
        if (item.product.category === "electronics") {  
            total *= 0.95;  
        }  
    }  
}
```

8. Feature Envy (2/2)

Solución:

```
//  Move Method al lugar correcto
class CartItem {
  calculateTotal(): number {
    let total = this.product.price * this.quantity;
    if (this.product.category === "electronics") total *= 0.95;
    return total;
  }
}

class CartCalculator {
  calculateItemTotal(item: CartItem): number {
    return item.calculateTotal();
  }
}
```

Herramientas de Detección (1/2)

Este proyecto usa: eslint-plugin-sonarjs

```
// eslint.config.js
export default tseslint.config([
  {
    extends: [
      sonarjs.configs.recommended, // SonarJS rules
    ],
    rules: {
      "sonarjs/cognitive-complexity": ["error", 15],
      "sonarjs/no-duplicate-string": "error",
      "sonarjs/no-identical-functions": "error",
    },
  },
]);
```

Herramientas de Detección (2/2)

SonarJS detecta automáticamente:

- **Cognitive Complexity** (complejidad cognitiva): ≤ 15
- **Code Duplication** (duplicación de código)
- **Identical Functions** (funciones idénticas)
- **Magic Numbers, Long Functions**, etc.

Ejecución:

```
pnpm lint # Analiza todo el proyecto
```

Tests como Red de Seguridad (1/2)

Safety Net (red de seguridad): Tests que garantizan que el refactoring no rompa funcionalidad

Ejemplo real (`src/__tests__/discount-calculations.test.ts`):

```
// Estos tests PASAN antes y después de refactorizar magic numbers
describe("calculateBulkDiscount", () => {
  it("should return 0 discount when quantity is less than 5", () => {
    const discount = calculateBulkDiscount(sampleItem, 1);
    expect(discount).toBe(0);
  });
});
```

Tests como Red de Seguridad (2/2)

```
it("should return 10% discount when quantity is exactly 5", () => {
  const discount = calculateBulkDiscount(sampleItem, 5);
  const expectedDiscount = sampleItem.price * 5 * 0.1;
  expect(discount).toBe(expectedDiscount);
});
// 11 tests más... todos pasan antes Y después
});
```

 **13 tests = red de seguridad completa**

Antes vs Después

Antes:

- Magic numbers: 12 ubicaciones
- Código duplicado: 8 ocurrencias
- Primitive obsession: 6 formatos
- Índice de mantenibilidad: 65

Después:

- Magic numbers: 0 (centralizados)
- Código duplicado: 1 (custom hooks)
- Primitive obsession: 0 (hooks)
- Índice de mantenibilidad: 89

Mejores Prácticas de Prevención

1. **Code Reviews:** Detectar smells temprano
2. **Automated Analysis:** ESLint, SonarQube en CI/CD
3. **Regular Refactoring:** Boy Scout Rule
4. **TDD:** Previene muchos smells naturalmente
5. **Pair Programming:** Dos ojos ven más

Boy Scout Rule

“Deja el código más limpio de como lo encontraste”

Principio: Cada vez que tocas código, mejóralo aunque sea un poco

Ejemplos:

- Renombrar variable confusa
- Extraer magic number a constante
- Añadir comentario útil

- Eliminar código muerto

Impacto: Mejora continua sin grandes refactorings

Ejercicio 1: Detectar con ESLint + SonarJS

Prompt:

Actúa como un desarrollador usando análisis estático para detectar code smells

CONTEXT0: Code smells son señales superficiales de problemas de diseño más profundos. NO son bugs (código funciona), pero afectan mantenibilidad futur
eslint-plugin-sonarjs detecta automáticamente: Cognitive Complexity (≤ 15), Duplicate Strings, Identical Functions, Magic Numbers. ESLint analiza código sin ejecutarlo (análisis estático).

TAREA: Ejecuta ESLint con SonarJS para detectar code smells automáticamente

PREPARACIÓN:

1. Cambiar a branch con smells intencionales: `git checkout refactor/01-smell`
2. Este branch tiene smells INTENCIONALMENTE agregados para el ejercicio

EJECUCIÓN:

- Comando: `pnpm lint`
- ESLint analizará todo el proyecto
- SonarJS reportará ubicación, severidad y tipo de smell

ANÁLISIS REQUERIDO:

Para cada smell reportado, documentar:

1. Archivo y línea exacta
2. Tipo de smell (cognitive-complexity, duplicate-string, etc.)
3. Severidad (error vs warning)
4. Código problemático

SHELLS ESPERADOS:

- Magic Numbers: valores hardcoded sin nombres (5, 0.1, 100, 0.15)
- Duplicate Strings: textos repetidos múltiples veces
- Cognitive Complexity: funciones con lógica compleja (> 15)

VALIDACIÓN: El comando debe reportar varios errores/warnings

Aprende: Herramientas automatizadas detectan smells sin esfuerzo

Ejercicio 2: Análisis Manual de Smells

Prompt:

Actúa como un code reviewer senior analizando code smells manualmente.

CONTEXT0: Análisis manual complementa herramientas automatizadas. Taxonomía de Code Smells: Structural (Long Method, Large Class), Behavioral (Duplicate Code, Dead Code), Object-Oriented (Feature Envy), Data (Magic Numbers, Primitive Obsession). Martin Fowler define criterios reconocidos. React antipatterns incluyen: componentes grandes, lógica duplicada, props drilling

TAREA: Crea documento code-smells-analysis.md con análisis manual completo.

BRANCH:

- Usar: refactor/01-smells (tiene smells intencionales)

ESTRUCTURA DEL DOCUMENTO:

Code Smells Analysis

Metodología

- Fecha: [fecha actual]
- Scope: src/components/, src/features/
- Criterios: Martin Fowler (Refactoring book) + React antipatterns
- Herramientas: Inspección manual + ESLint/SonarJS

🚩 Code Smells Identificados

[NOMBRE DEL SMELL] - Severidad: 🔴/🟡/⚠️

****Ubicación:**** archivo.tsx:línea

****Código:****

Markdown en typescript

[snippet del código problemático]

****Problema:**** [qué está mal]

****Impacto:**** [consecuencias en mantenibilidad]

****Refactor sugerido:**** [solución propuesta]

SMELLS A BUSCAR:

1. Magic Numbers (5, 0.1, 100, 0.15 sin nombres)
2. Duplicate Code (validación cantidad, formateo precio)
3. Primitive Obsession (formateo repetido de precios)
4. Long Parameter List (>3 parámetros)
5. Dead Code (imports no usados, funciones no llamadas)

ARCHIVOS PRIORITARIOS:

- src/features/shopping-cart/components/CartItem.tsx
- src/features/shopping-cart/components/CartSummary.tsx
- src/features/product-catalog/components/ProductCard.tsx

VALIDACIÓN: Documento debe tener al menos 3 smells documentados

Aprende: Análisis manual + automático = visión completa

Puntos Clave

1. **Code Smells:** Señales de alarma, no bugs
2. **Detection** (detección): Herramientas automatizadas + reviews
3. **Prioritization** (priorización): Impacto vs Esfuerzo
4. **Safety** (seguridad): Tests como red de seguridad
5. **Prevention** (prevención): Prácticas que evitan smells
6. **Culture** (cultura): Sensibilidad hacia diseño limpio